Working with climate data

Ellen Brock

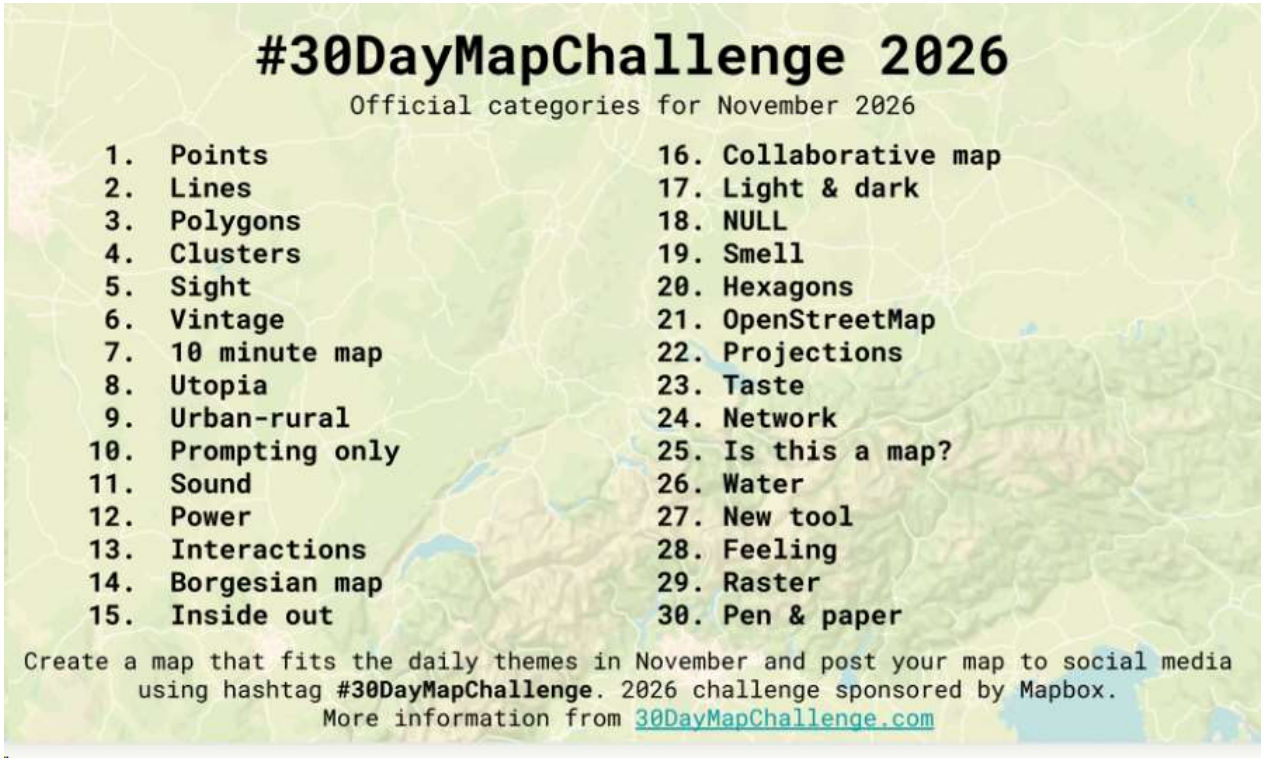
My research/work

- Economist by training (KULeuven)
- Working with Python, Google Earth Engine, GDAL, QGIS, OSMNX, etc
- Research work:
  - Land Surface Temperature (LST)
  - LST and indicators and street level, greenery, building, etc indicators
  - New research ongoing
- Consultancy work:
  - Deep learning
  - Machine learning
  - Landuse and land classification
  - Climate data (agritech)
  - Technical review (GEE mainly, QGIS India website, etc) - voluntary
- Learning newer tools:
  - GeoLibre
  - Dask (parallelization) using the kind of data (cloud native) in this presentation

Motivation 1

- Cloud-native workflows
- xarray, cloud native, pystac, STAC catalogues, geoparquet, zarr, cog, etc. Sounds new?
- The number of data options is large. It can be daunting!
- This material in this talk is work in progress (just my own learning)
- This talk will mainly focus on raster/gridded datasets (climate)
- How to access SOME data and do some visualisation
- Github link for scripts
- COMMENTS WELCOME!

Motivation 2: 30 day map challenge is going on …



Source: https://30daymapchallenge.com/#2026-themes

IT IS FREE!!

Good to learn QGIS, Python, new datasets, projections, Python modules, …

Participated in the 2024-2025 challenge

2025 quite a bit of topics on climate, less this year

2026 challenge is still going on…

Agenda

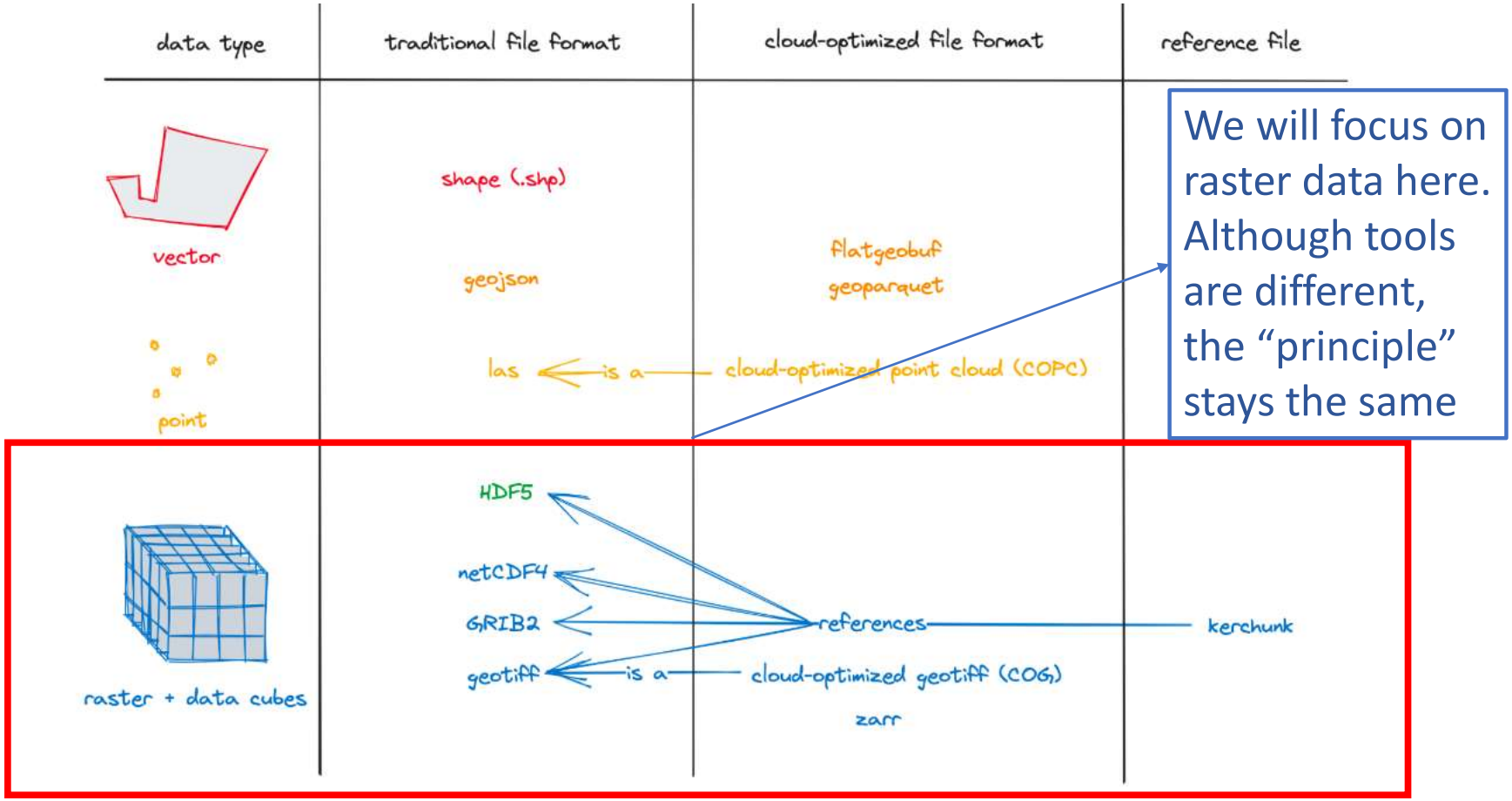
1. Introduction
2. Old way – new way
3. Examples
4. Some visualisation
5. What next?

Python

- All examples are done in Google Colab
- You can find the code also here on my GitHub
- All you need is a gmail for Google Colab
- You can replicate this also on your local computer too
  - You have to create to install Anaconda or mini conda
  - Install packages etc

Some climate datasets (not exhaustive!)

Datasets	Content
ERA5/ECMWF - Climate Data Store (ECMWF)	Temperature, pressure, wind, etc.
NASA Power	Energy and infrastructure
CHIRPS	Rainfall
IMD	Indian temperature, rainfall and station observations
Nasa Earth Data - LINK	Land & Vegetation, Ocean, etc.
CMIP6 / ESGF - Coupled Model Intercomparison Project	Climate simulations and future projections
Other like Sentinel-3, Landsat, etc	Next to regular bands (red, green, NIR), thermal bands (for LST)
NOAA	Weather, climate, ozone, ..
And many others …	

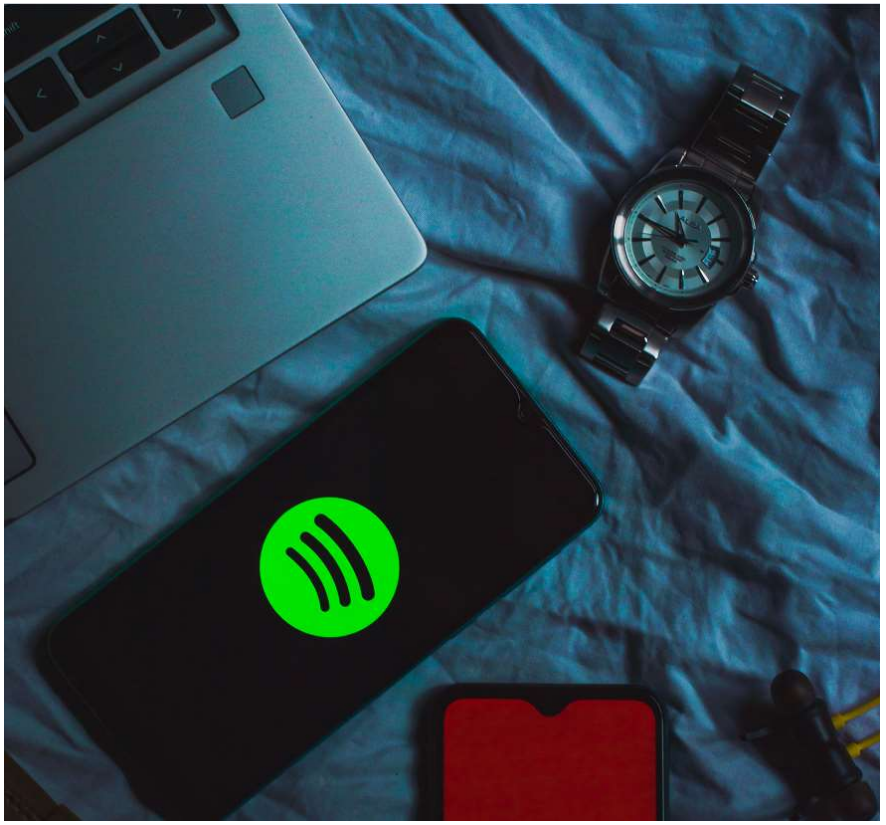


Source: https://guide.cloudnativegeo.org/

Analogy



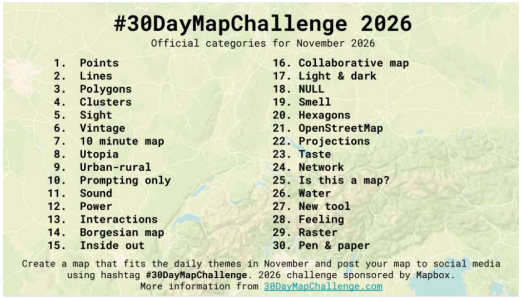
Source: Pexels, Giovanni Palma



Source: Pexels, walk.snap

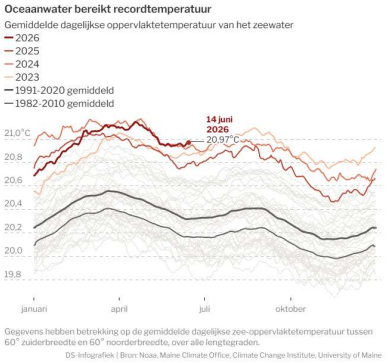
All ways (good or old data formats) lead to xarray (Python)

- The “access” can be little different across sources (.cog, .zarr, etc.) the data will require you to work with xarray
- Good way to start is with climate data (e.g. temperature, wind, etc)
- Think of something you like to work on, something you read in the newspaper, 30MapChallenge* and/or what you see in your course
- DO A QUICK TUTORIAL MAX (xarray in 45 min) and get going for xarray









Source:pexels 1 - Cemrecan Yurtman, 30DayMapChallenge, pexels 2 Yan Krukau, De Standaard news article.

* It will be challenging to all 30 days in xarray. Topic may also not always be related to climate.

Downloading example (“older format”)

- Example of European Centre for Medium-Range Weather Forecasts (ECMWF)
  - ERA5 – European Reanalysis v5
- Understanding what changed and sometimes data/tutorials are still in older formats
- netcdf, grib and hdf5 are mainly for climate data (gridded) and this format can be found in other datasets too (NASA, CHIRPS, etc)
- .geotiff is mainly for earth observation/satellite data (Landsat, Sentinel datasets, …) - used to be rasterio but xarray can handle too

The “research” question …

Bengaluru

At 32.60C, Bengaluru records highest temperature in a decade in September

Indian Express, 13 Sep 2026

IMD officials said the city has been recording maximum temperatures of 31 and 32 degrees Celsius for over a week now.

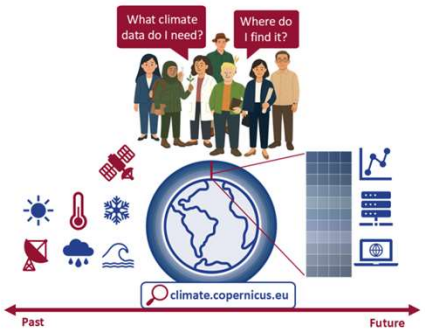
September's unusual heat takes a toll on Bengalureans

Patients are reporting severe dehydration, dizziness, headaches, light-headedness, muscle cramps and extreme fatigue. Prolonged exposure can lead to heat exhaustion and, in severe cases, heatstroke.

Deccan Herrald, 28 September 2026

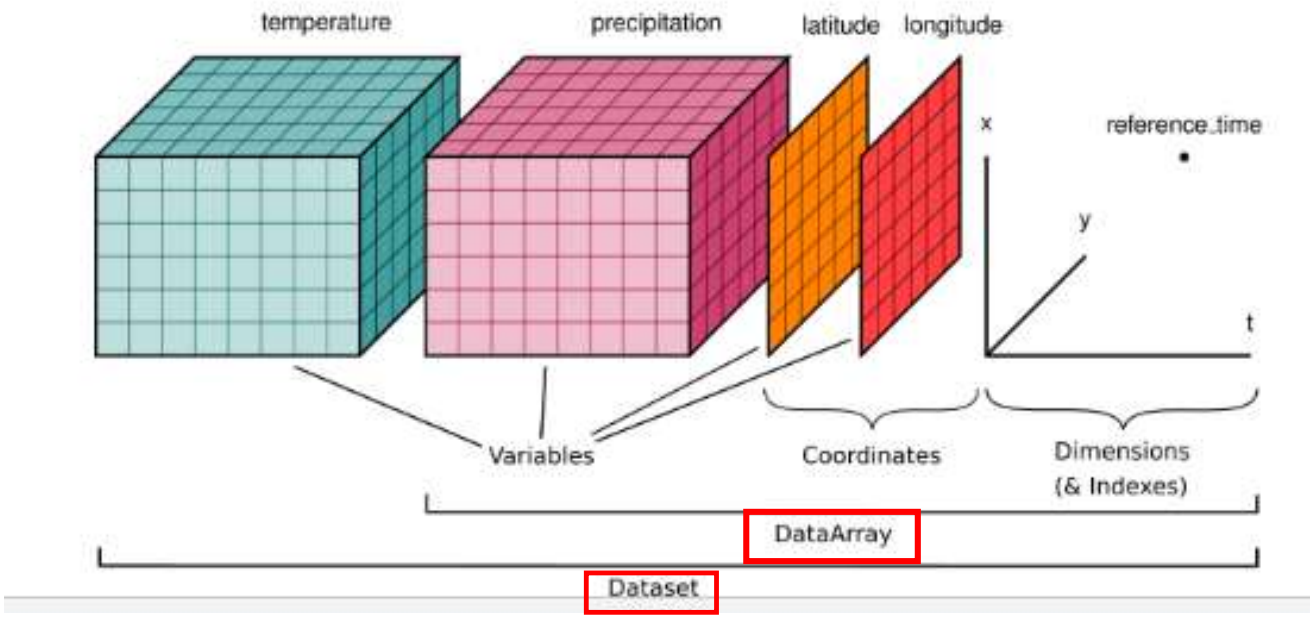
Xarray: Workhorse!!

- Xarray
  - Whether old or new dataset formats
  - And this for both for climate data or satellite images
- Temperature data is a good starting point to learn xarray
  - BUT LEARN TO ACCESS DATA (rather than in-built in tutorial)
- Lots of tutorials available (DON'T OVERDO!)
  - xarray
  - ECMWF (LINK 1, LINK 2, etc…)
  - Project Pythia
  - Other (see references)
- Then move to satellite images or other (Sentinel-2, Landsat, Digital Elevation, etc.)



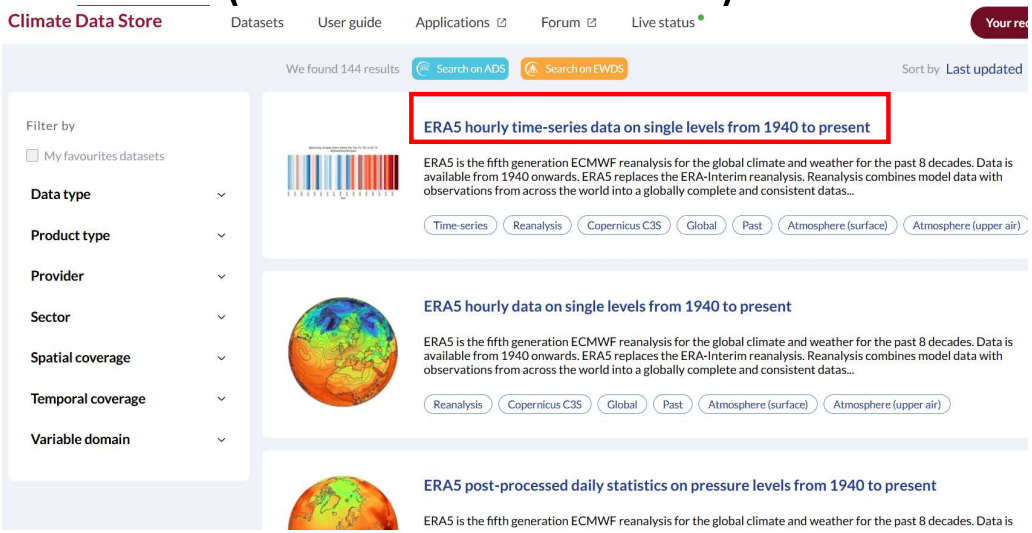
Source:
https://learning.ecmwf.int/enrol/index.php?id=117

Xarray: Dataset and data array



Climate Data Store (CDS)

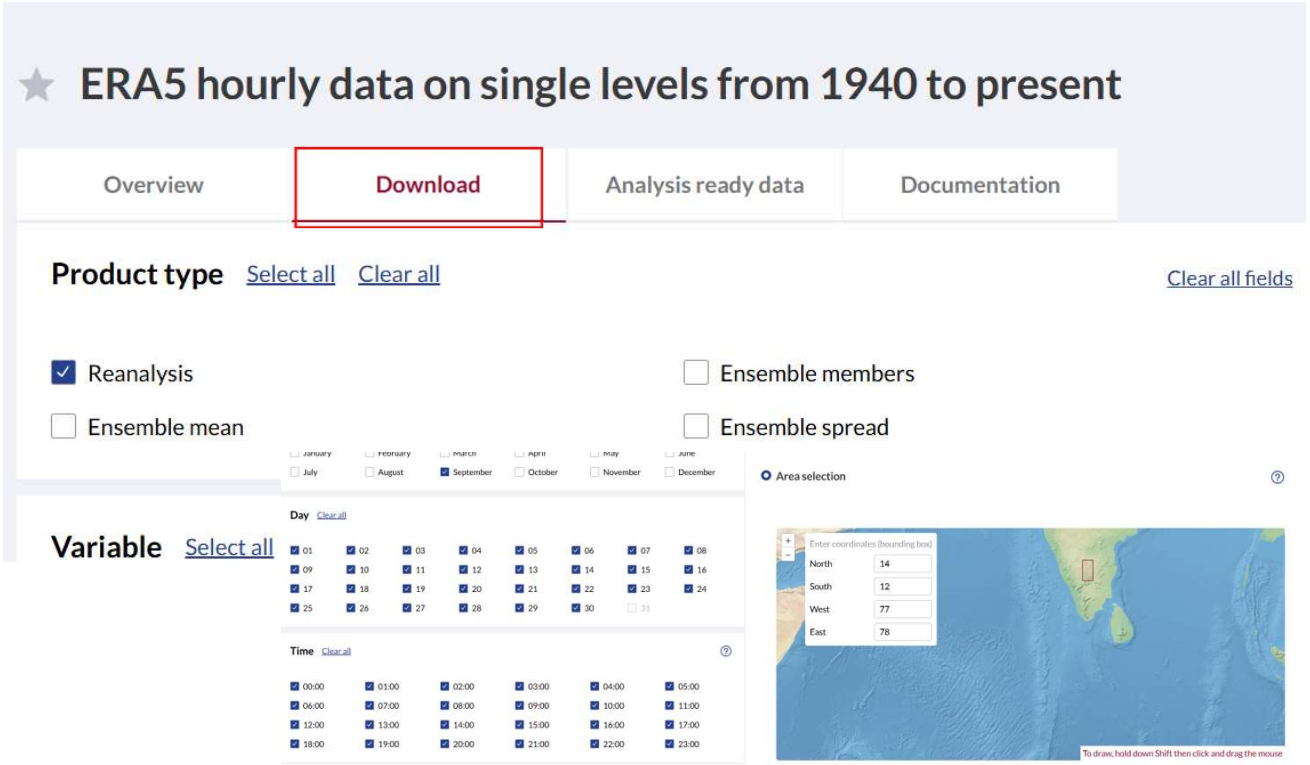
- Whether you work with the old (.nc) or new dataset formats (.zar), you need to register in the CDS and grab an API key
  - Details in the code
- Following example in tutorial HERE (with old .nc format)
- 1979 onwards



CDS datastore

- ERA5 hourly data on single levels from 1940 to present

Shows code to be pasted in Python

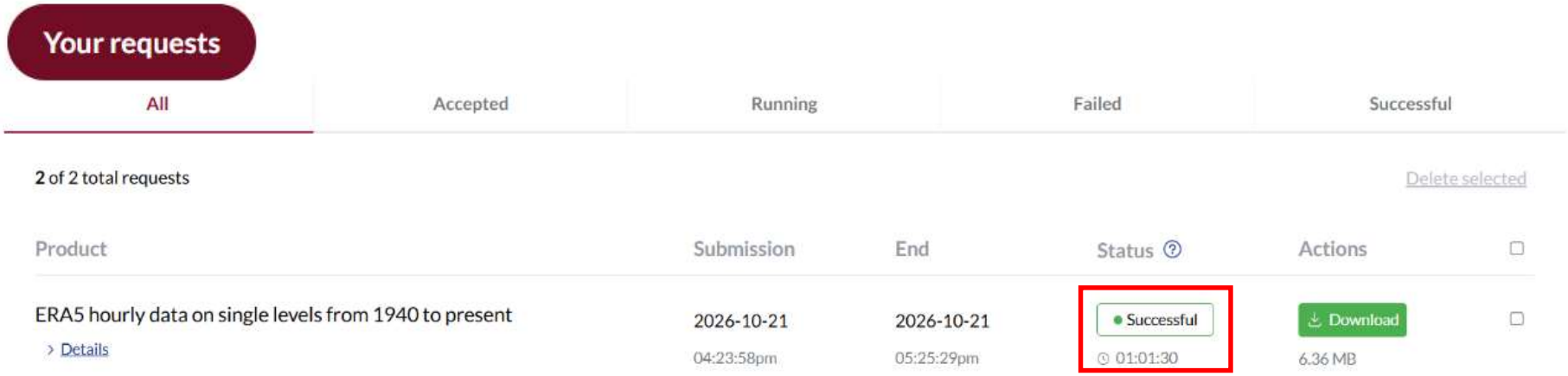



```
import cdsapi

dataset = "reanalysis-era5-single-levels"
request = {
    "product_type": ["reanalysis"],
    "variable": ["2m_temperature"],
    "year": [
        "1979", "1980", "1981",
        "1982", "1983", "1984",
        "1985", "1986", "1987",
        "1988", "1989", "1990",
        "1991", "1992", "1993",
        "1994", "1995", "1996",
        "1997", "1998", "1999",
        "2000", "2001", "2002",
        "2003", "2004", "2005",
        "2006", "2007", "2008",
        "2009", "2010", "2011",
        "2012", "2013", "2014",
        "2015", "2016", "2017",
        "2018", "2019", "2020",
        "2021", "2022", "2023",
        "2024", "2025", "2026"
    ],
    "month": ["09"],
    "day": [
        "01", "02", "03",
        "04", "05", "06",
        "07", "08", "09",
        "10", "11", "12",
        "13", "14", "15",
        "16", "17", "18",
        "19", "20", "21",
        "22", "23", "24",
        "25", "26", "27",
        "28", "29", "30"
    ],
    "time": [
        "00:00", "01:00", "02:00",
        "03:00", "04:00", "05:00",
        "06:00", "07:00", "08:00",
        "09:00", "10:00", "11:00",
        "12:00", "13:00", "14:00",
        "15:00", "16:00", "17:00",
        "18:00", "19:00", "20:00",
        "21:00", "22:00", "23:00"
    ],
    "data_format": "netcdf",
    "download_format": "unarchived",
    "area": [14, 77, 12, 78]
}

client = cdsapi.Client()
client.retrieve(dataset, request).download()
```

It took one hour to download – CDS dashboard



```
# Open an xarray dataset
ds = xr.open_dataset(file_path)
```

We can inspect the newly downloaded dataset. You can click on the different "buckets" on the right to expand the data.

```
ds
```

DATASET

```
<xarray.Dataset> Size: 25MB
Dimensions:    (time: 34560, expver: 2, latitude: 9, longitude: 5)
Coordinates:
  * time       (time) datetime64[ns] 276kB 1979-09-01 ... 2026-09-30T23:00:00
  * expver     (expver) int32 8B 1 5
  * latitude   (latitude) float32 36B 14.0 13.75 13.5 13.25 ... 12.5 12.25 12.0
  * longitude  (longitude) float32 20B 77.0 77.25 77.5 77.75 78.0
Data variables:
    t2m        (time, expver, latitude, longitude) float64 25MB ...
Attributes:
    Conventions:  CF-1.6
    history:      2026-10-21 11:55:08 GMT by grib_to_netcdf-2.42.0: grib_to_n...
```

```
da_temporary = ds['t2m']
```

DATA ARRAY

```
## You can see that we have a DataArray now.
da_temporary
```

```
<xarray.DataArray 't2m' (time: 34560, expver: 2, latitude: 9, longitude: 5)> Size: 25MB
[3110400 values with dtype=float64]
Coordinates:
  * time       (time) datetime64[ns] 276kB 1979-09-01 ... 2026-09-30T23:00:00
  * expver     (expver) int32 8B 1 5
  * latitude   (latitude) float32 36B 14.0 13.75 13.5 13.25 ... 12.5 12.25 12.0
  * longitude  (longitude) float32 20B 77.0 77.25 77.5 77.75 78.0
Attributes:
    units:      K
    long_name:  2 metre temperature
```

This dataset has only 1 variable since we asked only for one.

Data are now in Google Colab (or local Python)

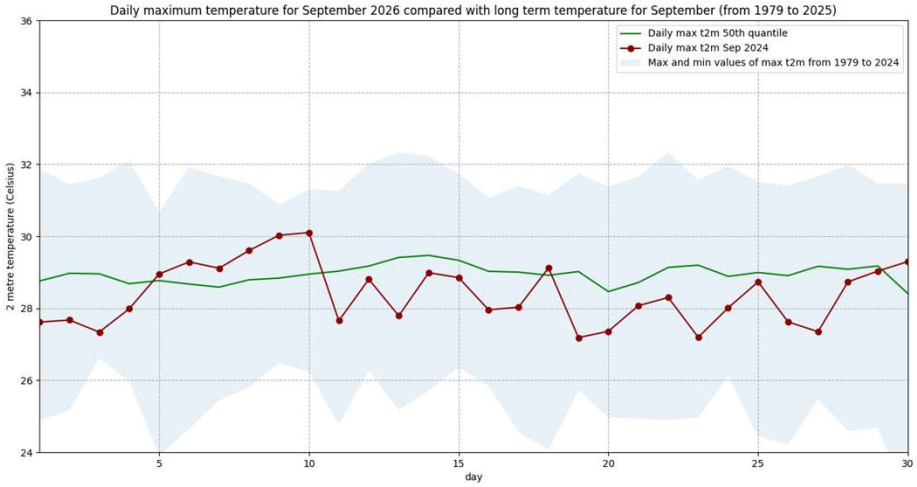
expver – when data is recent (provisionary version)

```
bangalore_2026_max = bangalore_sep_t2m_2026.groupby('time.day').max('time')
bangalore_2026_max
```

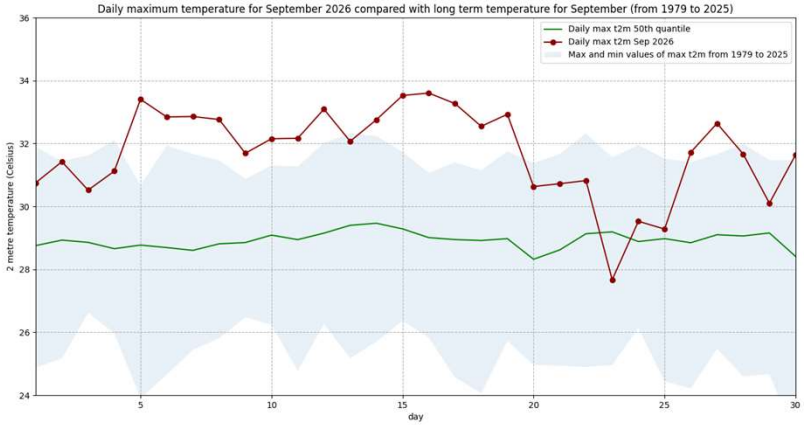
```
<xarray.DataArray 't2m' (day: 30)> Size: 240B
array([30.74997498, 31.42071661, 30.51669612, 31.12100986, 33.40229317,
       32.8434369 , 32.85581096, 32.76150629, 31.68908597, 32.15042673,
       32.16544301, 33.08999322, 32.06842347, 32.75419467, 33.52483065,
       33.60229905, 33.26722519, 32.54529331, 32.92624414, 30.63150482,
       30.72100662, 30.82047248, 27.66880736, 29.52704442, 29.27850822,
       31.72161884, 32.63902305, 31.66312155, 30.10127031, 31.64086894])
Coordinates:
  * day      (day) int64 240B 1 2 3 4 5 6 7 8 9 ... 22 23 24 25 26 27 28 29 3(
Attributes:
    units:      K
    long_name:  2 metre temperature
```

After some more code:

2025



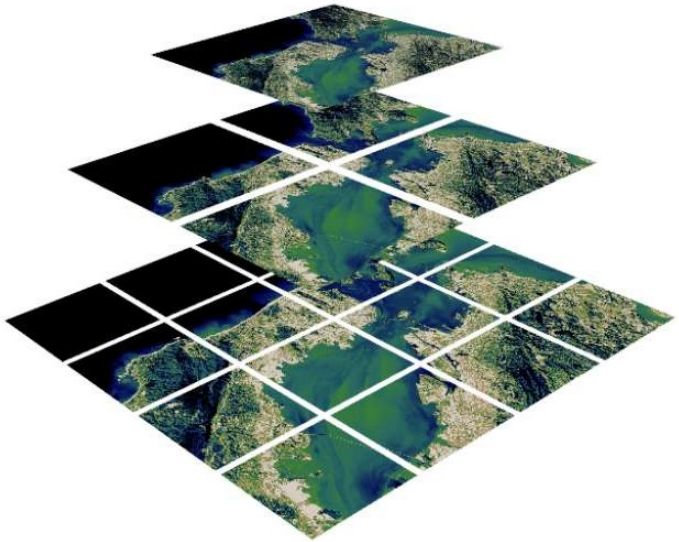
2026



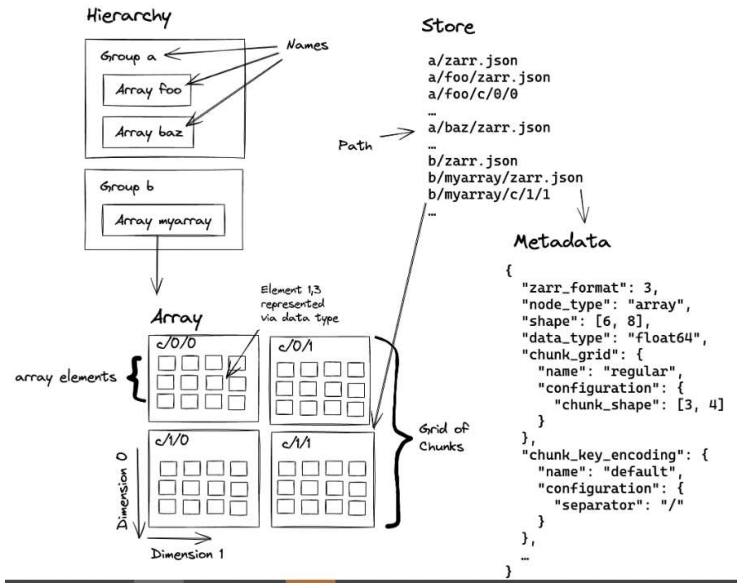
Source: Own computation based on tutorial [ECMWF tutorial]

COG/ZARR – New formats

COG



ZARR



- Meta data in one read
- “Query” the meta data
- Meta data points you to what part you will need
- Think time, latitude, longitude, elevation, spectral bands, etc.
- Chunks in the form of small tiles, internal tiles,
- Lazy compute
- Unless .load or .compute in xarray you will not “download” the data
- ZARR: (time, x, y, chunks, bands, etc)
- Standard more uniform across providers

Source: https://guide.cloudnativegeo.org/overview.html#/what-are-cogs and https://tutorial.xarray.dev/intermediate/intro-to-zarr.html

Sentinel-2 data can exist both in COG (pystac and this from 2 entry points) and ZARR (EOPF)

ECMWF zarr format

- ECMWF now moving to Analysis-Ready Cloud-Optimised (ARCO) Zarr LINKEDIN post (~ 4 months ago).
- Two ways it is stored

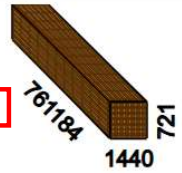
	Better for	t2m.encoding["chunks"] Python script (time steps, longitude, latitude	**Code**
Time-chunked	Large area for a short time period	(1, 721, 1024)	LINK
Geo-chunked	Small area for large time period	(67584, 4, 8)	LINK

- Data is from 1950 onwards

```
t2m = ds["t2m"]
t2m
```

xarray.DataArray 't2m' (**time**: 761184, **latitude**: 721, **longitude**: 1440)

	Array	Chunk
Bytes	2.88 TiB	4.12 MiB
Shape	(761184, 721, 1440)	(67584, 4, 4)
Dask graph	781920 chunks in 2 graph layers	
Data type	float32 numpy.ndarray	



▾ Coordinates:

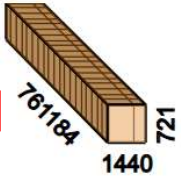
time	(time)	datetime64[ns]	1940-01-01 ... 2026-10-31T23:00:00
latitude	(latitude)	float64	-90.0 -89.75 -89.5 ... 89.75 90.0
longitude	(longitude)	float64	-180.0 -179.8 ... 179.5 179.8

GeoChunked: Long time series

```
t2m = ds["t2m"]
t2m
```

xarray.DataArray 't2m' (**time**: 761184, **latitude**: 721, **longitude**: 1440)

	Array	Chunk
Bytes	2.88 TiB	2.82 MiB
Shape	(761184, 721, 1440)	(1, 721, 1024)
Dask graph	1522368 chunks in 2 graph layers	
Data type	float32 numpy.ndarray	



▾ Coordinates:

time	(time)	datetime64[ns]	1940-01-01 ... 2026-10-31T23:00:00
latitude	(latitude)	float64	-90.0 -89.75 -89.5 ... 89.75 90.0
longitude	(longitude)	float64	-180.0 -179.8 ... 179.5 179.8

▸ Attributes: (30)

TimeChunked: Bigger areas

Source: Own computations

Geochunked data - CODE

```
## Geo-chunked data for access along the time dimension (e.g. for time-series at a single point)
geochunked_sfc_url = "https://arco.datastores.ecmwf.int/cadl-arco-geo-002/arco/reanalysis_era5_single_levels/sfc/geoChunked.zarr"
```

```
# Open the zarr store with xarray, users must insert their API key where indicated.
ds = xr.open_zarr(
    geochunked_sfc_url,
    consolidated=True,
     storage_options = {
        "headers": {"Authorization": f"Bearer {cdsapi_key}"}
    }
)

# Inspect the variables
ds
```

63 TB

```
<xarray.Dataset> Size: 63TB
Dimensions:    (time: 761184, latitude: 721, longitude: 1440)
Coordinates:
  * time       (time) datetime64[ns] 6MB 1940-01-01 ... 2026-10-31T23:00:00
  * latitude   (latitude) float64 6kB -90.0 -89.75 -89.5 ... 89.5 89.75 90.0
  * longitude  (longitude) float64 12kB -180.0 -179.8 -179.5 ... 179.5 179.8
Data variables: (12/20)
    blh        (time, latitude, longitude) float32 3TB dask.array<chunksize=(67584, 4, 4), meta=np.ndarray>
    cbh        (time, latitude, longitude) float32 3TB dask.array<chunksize=(67584, 4, 4), meta=np.ndarray>
    d2m        (time, latitude, longitude) float32 3TB dask.array<chunksize=(67584, 4, 4), meta=np.ndarray>
```

You keep on “querying” the metadata

Put into a data array:

```
t2m = ds["t2m"]
t2m
```

```
<xarray.DataArray 't2m' (time: 761184, latitude: 721, longitude: 1440)> Size: 3TB
dask.array<open_dataset-t2m, shape=(761184, 721, 1440), dtype=float32, chunksize=(
```

3 TB

Keep on narrowing to what you want (Bangalore, September only and from 1979 only). At some point

```
t2m_bangalore = t2m_september.sel(
    longitude=slice(77, 78),
    latitude=slice(12, 14)
)
```

Some more “queries” till you actually load. This is “lazy evaluation”. Then you call the data by load() or compute():

```
%time
t2m_bangalore = t2m_bangalore.load()
t2m_bangalore
```

```
t2m_bangalore = t2m_september.sel(
    longitude=slice(77, 78),
    latitude=slice(12, 14)
)
```

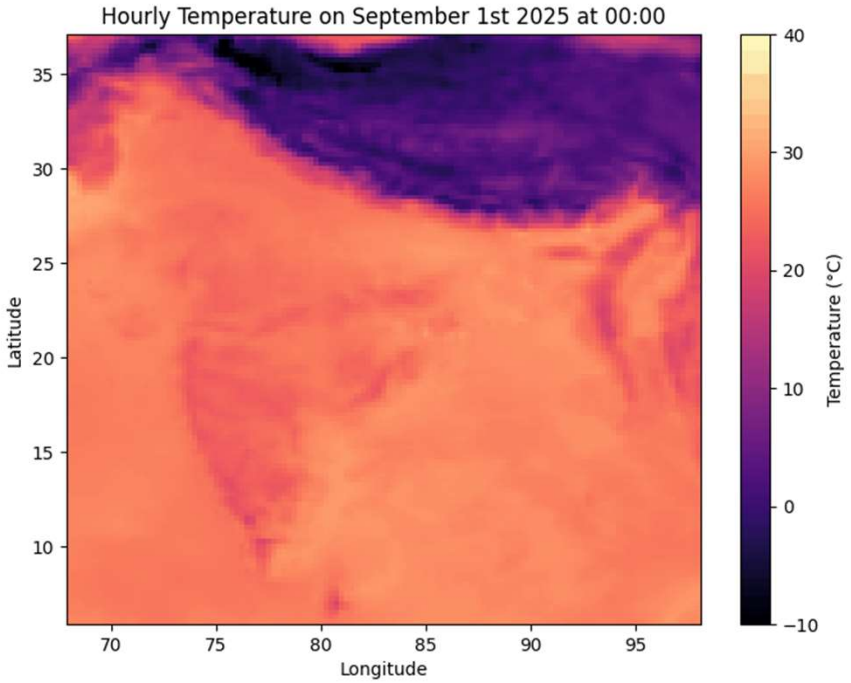
```
t2m_bangalore
```

```
<xarray.DataArray 't2m' (time: 34560, latitude: 9, longitude: 5)> Size: 6MB
```

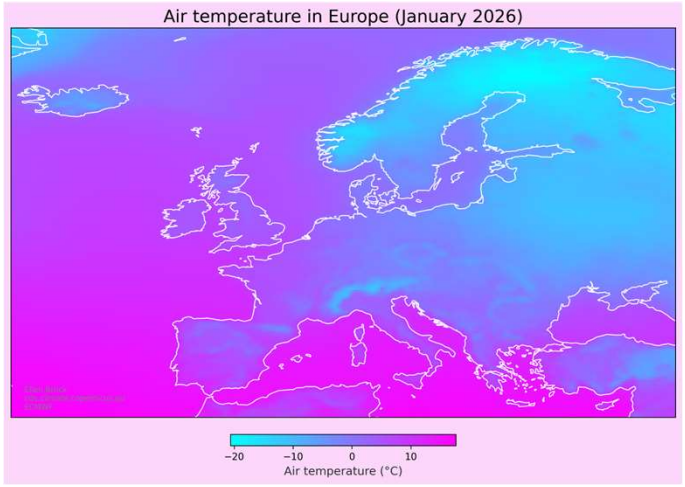
6 MB

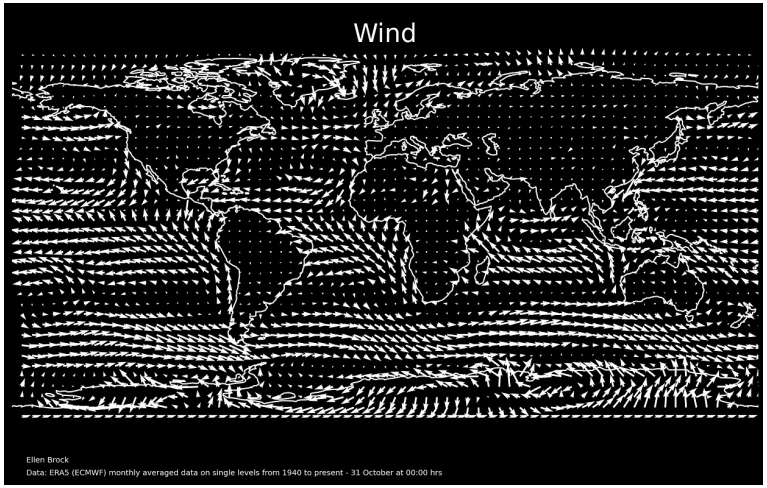
Timechunked data - CODE

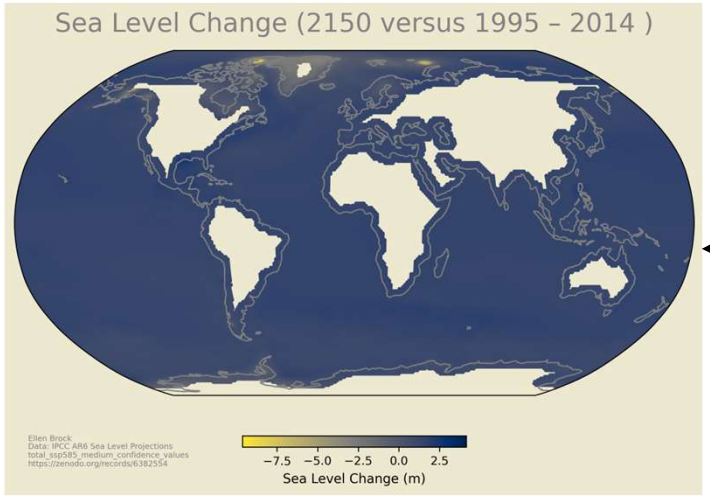
- Take the TimeChunked data (larger area/shorter time period)
- CODE for an animation



Some more fun: 30DayMapChallenge 2025





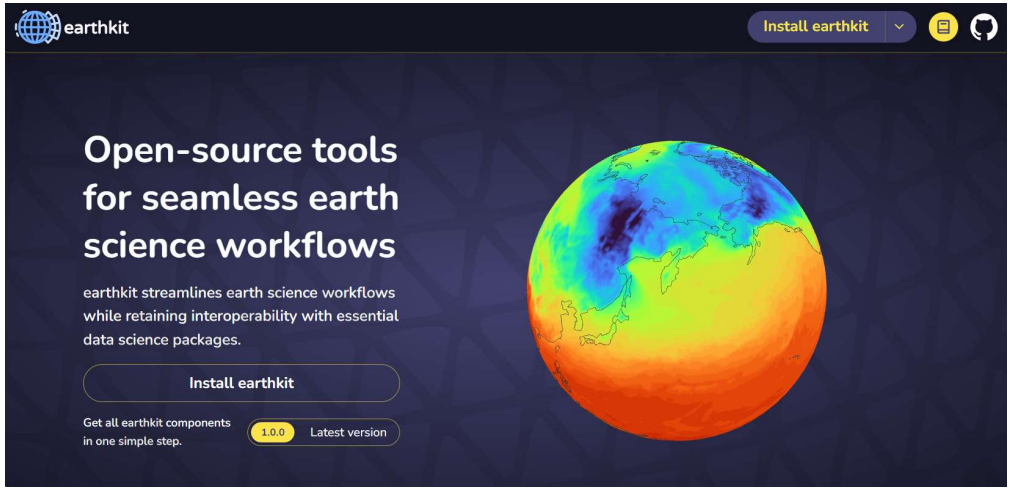


- This data was downloaded the old way through Zenodo
- Coupled Model Intercomparison Project Phase 6 (CMIP6)
- Tutorial for newer (zarr formats)
- Exercise: Can you find easier ways than Zenodo like zar format or so?

Source: Own computations
CODE

Earthkit

- Plotting and downloading even made easier (ECMWF initiative)



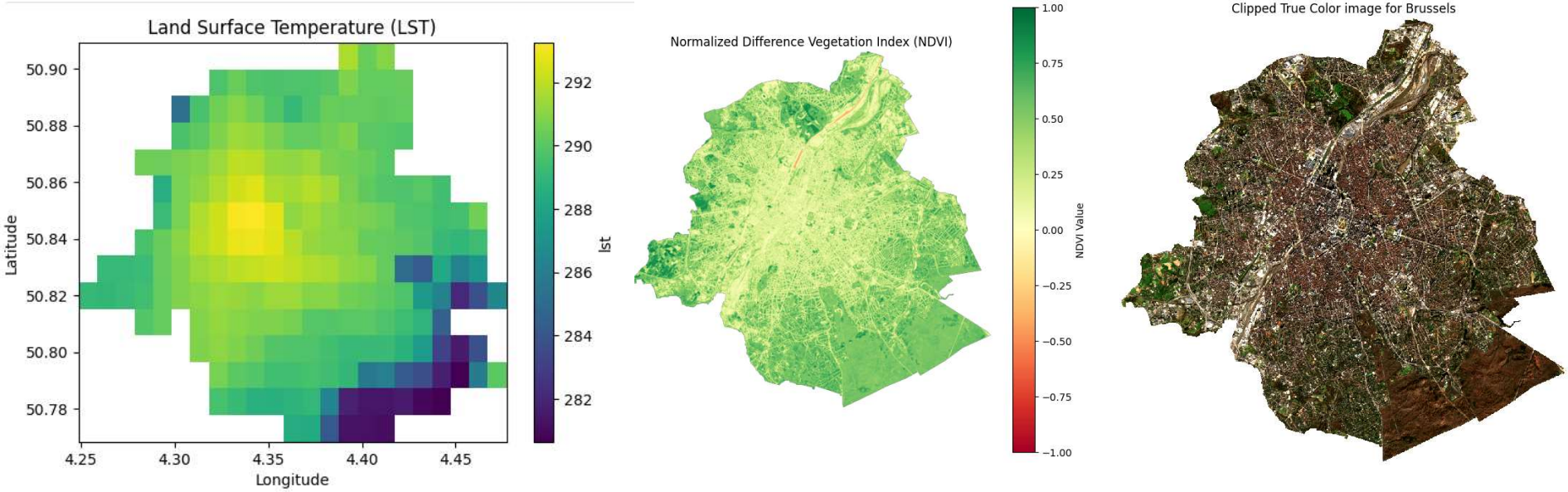
Source: https://earthkit.ecmwf.int/

- Gallery
- Builds on top of xarray
- Tutorials – earthkit not tried yet
- Emma Pidduck (ECMWF) tutorials during the 30DayMapChallenge 2026



EOPF – Land surface temperature (LST)

- European Space Agency's (ESA) Earth Observation Processor Framework (EOPF) initiative.
- Sentinel data in ZARR format (Brussels, Belgium)



Source: Own computations from the EOPF community notebook competition

Alternative – ERA5 on Google Cloud, Planetary computer, weather bench

Dataset	Period	Format t2m.encoding["chunks"]	Use
Weather Bench – Exercise for you!*	1959 to early 2023	(8, 240, 121) - each day every 6 hours observation so two days	A mix!
Google - **ARCO-ERA5**	1940-2026	(1, 721, 1440) - each chunk is **one hour, whole globe - zarr**	Large areas, short time periods
Planetary computer (pystac) - CODE	1979–2020	(372, 150, 150) – One zarr dataset is one month of a year – within each month-year hourly data each year	A mix !

* See link in the second source to get started
- Based on sources: https://www.technicalgis.com/tutorials/era5-python-xarray/ and

No sign-in/sign up (no token) required for all these datasets

I would work with ARCO from the ECMWF

STAC specification

- Catalog: Contains several collections – enter there (Planetary Computer, Earth Search, etc.
  - Collection: Sentinel, Landsat, Digital Elevation, etc
    - Item: A single file or asset (e.g. band (red, green, etc) of a Sentinel-2 scene at a particular time
- Where can you find these catalogs?

- More information: Cloud-Optimized Geospatial Formats Guide

STAC: Spatial Temporal Asset catalogue

- https://stacindex.org/catalogs - ~ 150 catalogs
- Some well-known catalogs

	Content
Earth Search	Sentinel-1-2, Landsat, Digital Elevation Models (DEM)
Microsoft Planetary computer	150+ collections from DEM, Sentinel, climate data, era5 (ECWMF), etc
Source Cooperative	Not a STAC catalogue as such but platform that can contain catalogs
Other	

pystac example – Planetary computer

```
!pip install pystac-client planetary-computer --quiet
```

```
import pandas as pd
import planetary_computer
## import rioxarray as rxr

from pystac_client import Client
```

```
catalog = Client.open("https://planetarycomputer.microsoft.com/api/stac/v1", modifier=planetary_compute
```

```
collections = catalog.get_all_collections()

datasets = []
for col in collections:
    datasets.append({'collection': col.id, 'description': col.description})
df_col = pd.DataFrame(datasets)
df_col # If you uncomment this, you can easily browse the table in Google Colab
```

In Google Colab

pystac example

```
collections = catalog.get_all_collections()

datasets = []
for col in collections:
    datasets.append({'collection': col.id, 'description': col.description})
df_col = pd.DataFrame(datasets)
df_col # If you uncomment this, you can easily browse the table in Google Colab
```

	collection	description
0	3dep-lidar-classification	This collection is derived from the [USGS 3DEP...
1	3dep-lidar-copc	This collection contains source data from the ...
2	3dep-lidar-dsm	This collection is derived from the [USGS 3DEP...
3	3dep-lidar-dtm	This collection is derived from the [USGS 3DEP...
4	3dep-lidar-dtm-native	This collection is derived from the [USGS 3DEP...
...	...	...
135	terraclimate	[TerraClimate](http://www.climatologylab.org/t...
136	us-census	The [2020 Census](https://www.census.gov/progr...
137	usda-cdl	The Cropland Data Layer (CDL) is a product of ...
138	usgs-lcmap-conus-v13	The [Land Change Monitoring, Assessment, and P...
139	usgs-lcmap-hawaii-v10	The [Land Change Monitoring, Assessment, and P...

Source: Own code

38	era5-pds	ERA5 is the fifth generation ECMWF atmospheric reanalysis of the global climate covering the period from January 1950 to present. ERA5 is produced by the Copernicus Climate Change Service (C3S) at ECMWF. Reanalysis combines model data with observations from across the world into a globally complete and consistent dataset using the laws of physics. This principle, called data assimilation, is based on the method used by numerical weather prediction centres, where every so many hours (12 hours at ECMWF) a previous forecast is combined with newly available observations in an optimal way to produce a new best estimate of the state of the atmosphere, called analysis, from which an updated, improved forecast is issued. Reanalysis works in the same way, but at reduced resolution to allow for the provision of a dataset spanning back several decades. Reanalysis does not have the constraint of issuing timely forecasts, so there is more time to collect observations, and when going further back in time, to allow for the ingestion of improved versions of the original observations, which all benefit the quality of the reanalysis product. This dataset was converted to Zarr by [Planet OS] (https://planetos.com/). See [their documentation](https://github.com/planet-os/notebooks/blob/master/aws/era5-pds.md) for more. ## STAC Metadata Two types of data variables are provided: "forecast" (`fc`) and "analysis" (`an`). * An **analysis**, of the atmospheric conditions, is a blend of observations with a previous forecast. An analysis can only provide [instantaneous] (https://confluence.ecmwf.int/display/CKB/Model+grid+box+and+time+step) parameters (parameters valid at a specific time, e.g temperature at 12:00), but not accumulated parameters, mean rates or min/max parameters. * A **forecast** starts with an analysis at a specific time (the 'initialization time'), and a model computes the atmospheric conditions for a number of 'forecast steps', at increasing 'validity times', into the future. A forecast can provide [instantaneous](https://confluence.ecmwf.int/display/CKB/Model+grid+box+and+time+step) parameters, accumulated parameters, mean rates, and min/max parameters. Each [STAC](https://stacspec.org/) item in this collection covers a single month and the entire globe. There are two STAC items per month, one for each type of data variable (`fc` and `an`). The STAC items include an `ecmwf:kind` properties to indicate which kind of variables that STAC item catalogs. ## How to acknowledge, cite and refer to ERA5 All users of data on the Climate Data Store (CDS) disks (using either the web interface or the CDS API) must provide clear and visible attribution to the Copernicus programme and are asked to cite and reference the dataset provider: Acknowledge according to the [licence to use Copernicus Products](https://cds.climate.copernicus.eu/api/v2/terms/static/licence-to-use-copernicus-products.pdf). Cite each dataset used as indicated on the relevant CDS entries (see link to "Citation" under References on the Overview page of the dataset entry). Throughout the content of your publication, the dataset used is referred to as Author (YYYY). The 3-steps procedure above is illustrated with this example: [Use Case 2: ERA5 hourly data on single levels from 1979 to present] (https://confluence.ecmwf.int/display/CKB/Use+Case+2%3A+ERA5+hourly+data+on+single+levels+from+1979+to+present). For complete details, please refer to [How to acknowledge and cite a Climate Data Store (CDS) catalogue entry and the data published as part of it] (https://confluence.ecmwf.int/display/CKB/How+to+acknowledge+and+cite+a+Climate+Data+Store+%28CDS%29+catalogue+entry+and+the+data+published+as+part+of+it).

```
df_era5 = df_col[df_col["collection"].str.contains("era5", case=False, na=False)]
df_era5
```

	collection	description
38	era5-pds	ERA5 is the fifth generation ECMWF atmospheric...

```python
# long min, lat min, long max, lat max
## india_geometry = [77, 12,  78, 14]
eur_geometry = [-25, 34, 45, 72]
datetime_range = "2020-01-01/2020-12-31"

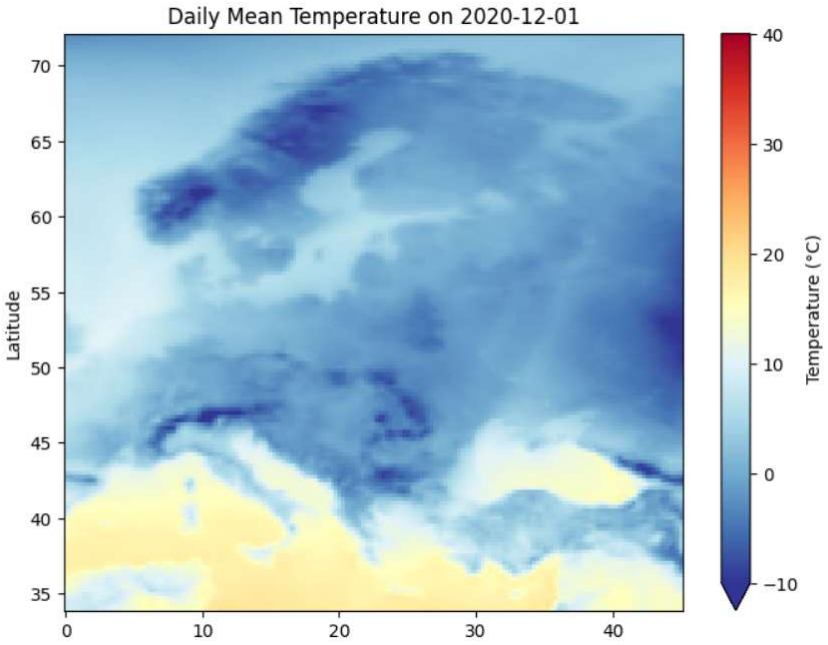
## Code obtained from:
## https://colab.research.google.com/github/developmentseed/ingarss-workshop-2024/blob/main/book/docs/01_stac_and_zarr.ipynb#scrollTo=80311144
## And:
## https://www.technicalgis.com/code/era5-python-xarray/
```

```python
search = catalog.search(
    collections=["era5-pds"],
    bbox=eur_geometry,
    datetime=datetime_range,
    query={
        "era5:kind": {
            "eq": "an"  # Filter for analysis data only - you also have "fc" for forecasts
        },
    }
)
```

```python
items = search.item_collection()
print(f"Number of items: {len(items)}")
```

```
umber of items: 12
```

Data are organized monthly if you do the search with a reference to the data of each month
LESS HANDY!!



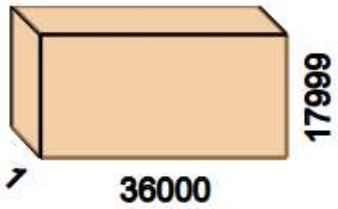
Lazy evaluation once more

- Sea Surface Temperature (is also in CDS (ECMWF) and other sources) but from SOURCE COOPERATIVE
  - Source Cooperative is not a STAC (SpatioTemporal Asset Catalog) itself but it is a cloud-native data publishing platform
- This example illustrates "Lazy evaluation" well: Unless you type "compute()" or "load()" nothing gets downloaded/loaded in memory
- Uses pystac (not the pystac client)
  - pystac for managing static STAC metadata (creating, reading, and writing local or cloud files), while pystac-client interacting with catalogs
- Looks like .json file – I am confused now ☺
    - path = "https://data.source.coop/ausantarctic/ghrsst-mur-v2/2025/02/09/20250209090000-JPL-L4_GHRSST-SSTfnd-MUR-GLOB-v02.0-fv04.1.stac-item.json"

Lazy evaluation contd …

xarray.DataArray 'analysed_sst' (**time**: 1, **latitude**: 17999, **longitude**: 36000)

	Array	Chunk
Bytes	1.21 GiB	1.21 GiB
Shape	(1, 17999, 36000)	(1, 17999, 36000)
Dask graph	1 chunks in 3 graph layers	
Data type	int16 numpy.ndarray	



▾ Coordinates:

time	(time)	datetime64[ns]	2025-02-09
latitude	(latitude)	float64	89.99 89.98 89.97 ... -89.98 -89.99
longitude	(longitude)	float64	-180.0 -180.0 ... 180.0 180.0
spatial_ref	()	int32	4326

▾ Attributes:

nodata : -32768

This is data for the entire world.

Trying to plot, Python (and Google Colab) will crash.

Took a smaller sample (Europe)

Please see script using the example from the dataset

Lazy evaluation contd …

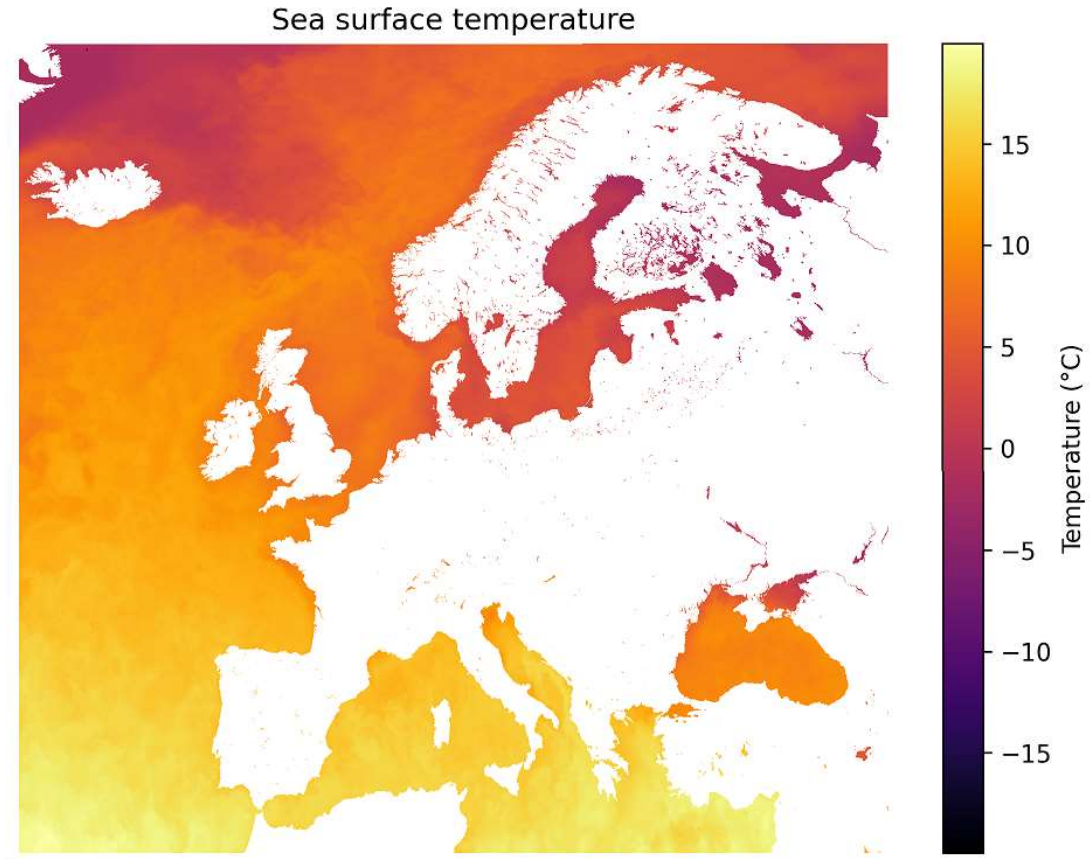
```
In [19]:
sst_europe = sst.sel(
    latitude=slice(72, 34),
    longitude=slice(-25, 45)
)
sst_europe

Out[19]: <xarray.Dataset> Size: 53MB
```

```
In [20]: sst_europe = sst_europe.compute()

In [22]: sst_europe

Out[22]: <xarray.Dataset> Size: 53MB
```

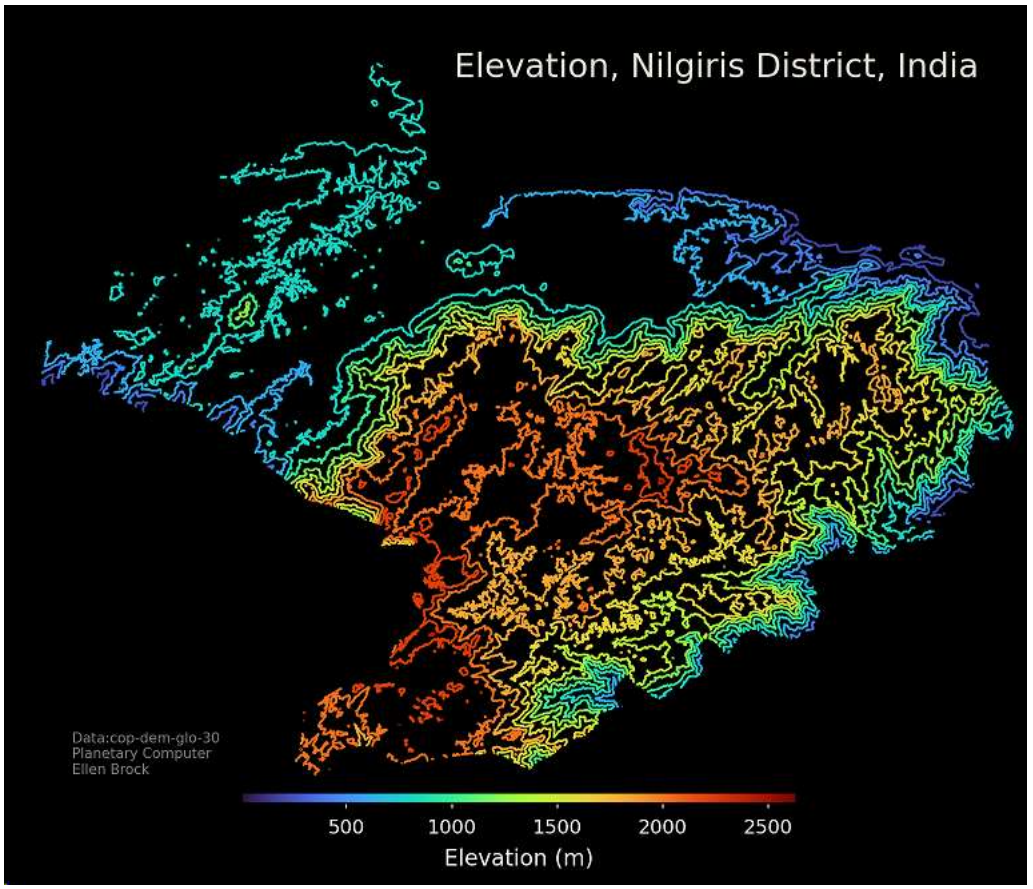


Source: Own computations - CODE

pystac-client example with DEM

- DEM: Digital Elevation model
- Using Datameet data (Indian districts)
- Planetary computer (for the DEM dataset)
- Planetary also good for Landsat data, etc.





Source: Own computations - CODE

Google zarr + cartopy

```
!pip install zarr gcsfs xarray cartopy matplotlib --quiet
```

```
import time, warnings
import numpy as np
import xarray as xr
import matplotlib.pyplot as plt

import cartopy.crs as ccrs
import cartopy.feature as cfeature

warnings.filterwarnings("ignore")
```

```
URL  = "gs://gcp-public-data-arco-era5/ar/full_37-1h-0p25deg-chunk-1.zarr-v3"
```

Of course, you apply cartopy to all other datasets (Planetary Computer, CDS, etc.)

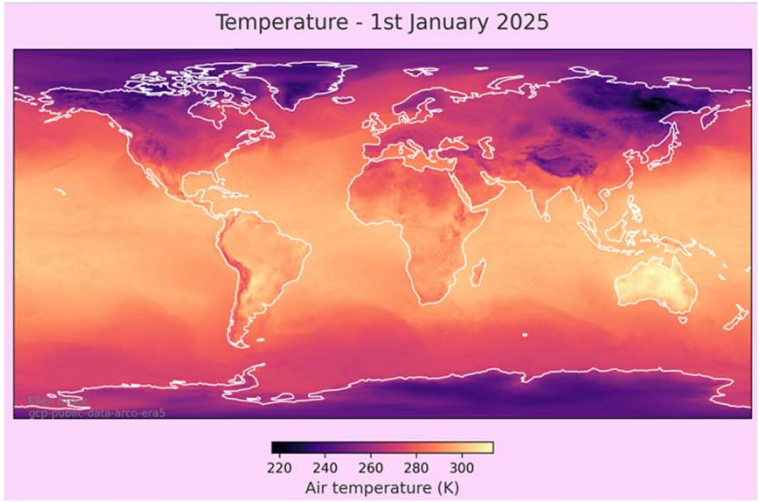
```
map_projection = ccrs.Mollweide() # projection on the map

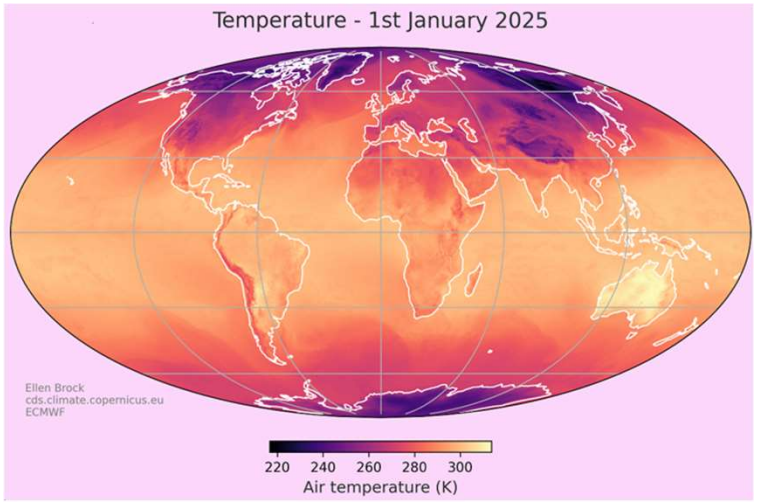
fig, ax = plt.subplots(figsize=(10, 8), subplot_kw={'projection': map_projection},
                       facecolor='#FCD7FB')

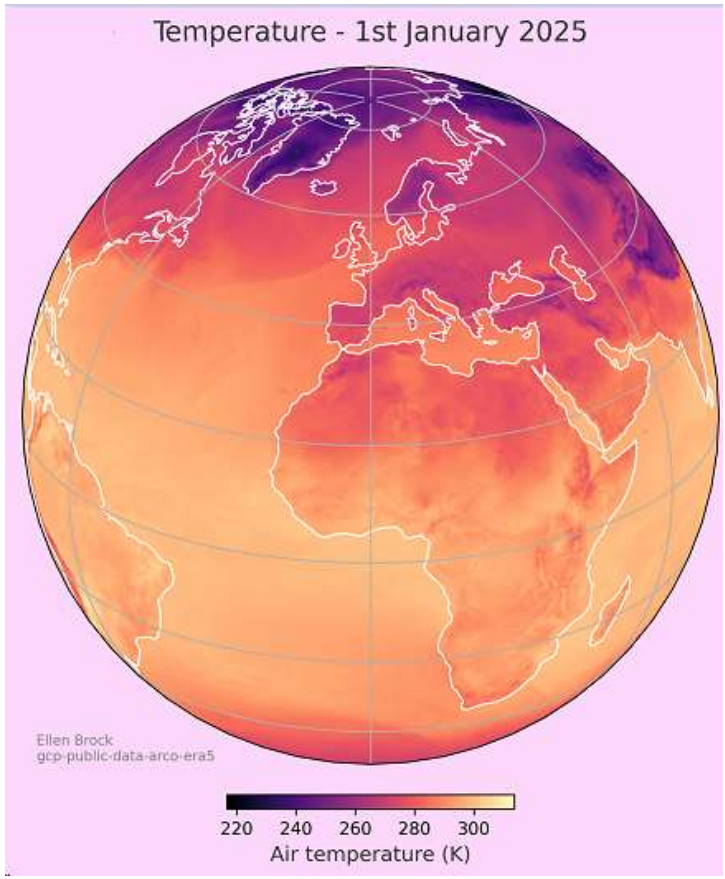
ax.coastlines(color = 'white')
ax.gridlines()

im = t2m.plot(
    ax=ax,
    x='longitude',
    y='latitude',
    transform=ccrs.PlateCarree(),  # Coordinates are in degrees lat/lon
    cmap='magma',
    add_colorbar=False
)
```

CODE

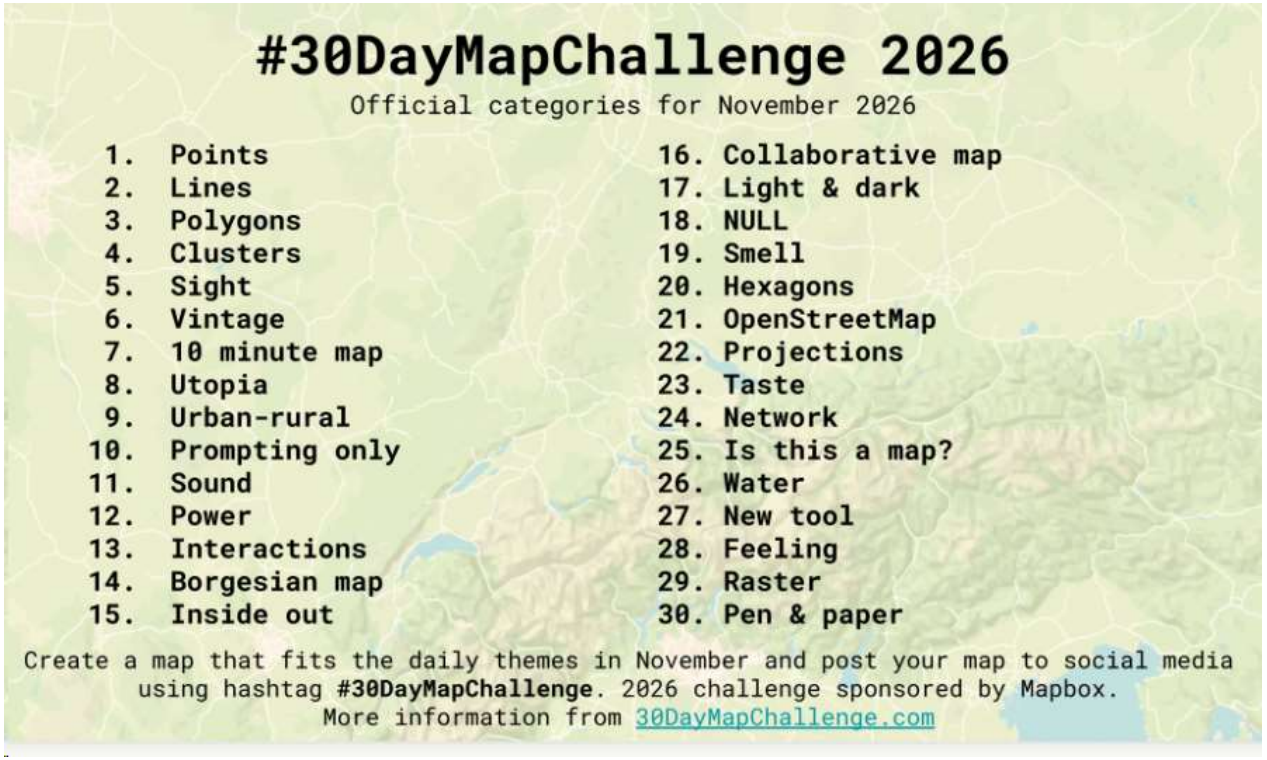






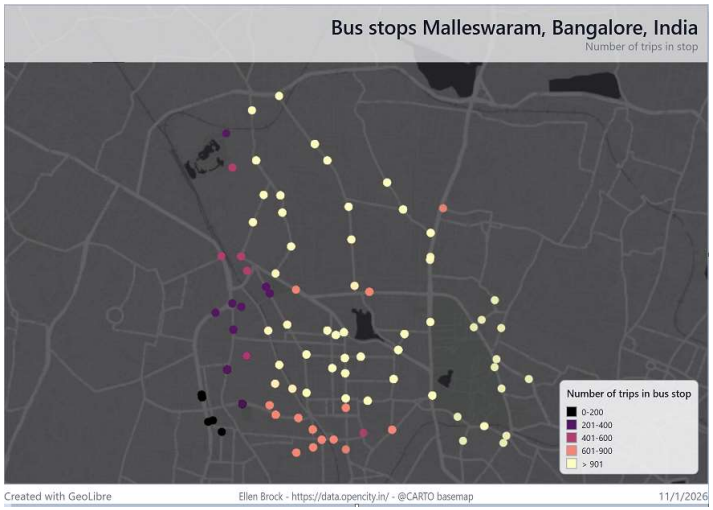
I will show these plots on Day 22 of the 2026 Map Challenge – Please if you use parts of the code, please attribute

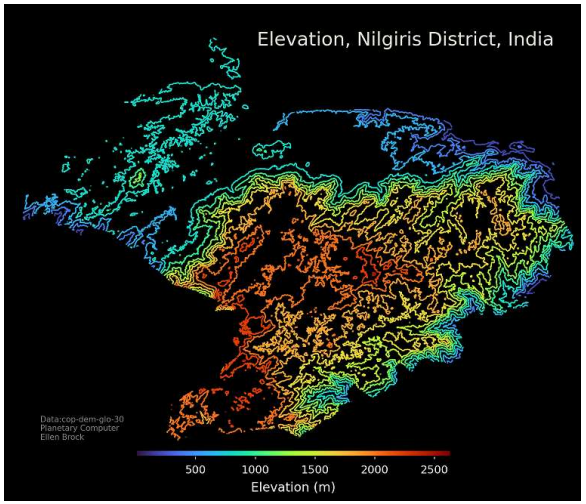
30 day map challenge is going on …



Source: https://30daymapchallenge.com/#2026-themes

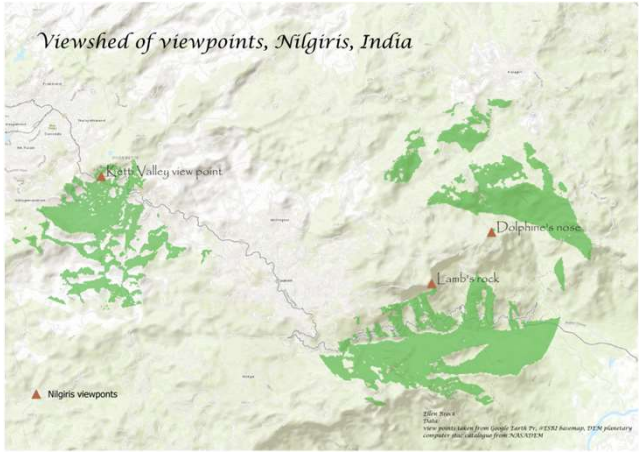
2026 Map Challenge











Twitter - #30DayMapChallenge
LinkedIn - #30DayMapChallenge/

1: GeoLibre
2: Python
3: Python
4: QGIS
5: QGIS

Source: Own computations

What this talk did not cover

- We can also work with xarray using satellite data (instead of weather data) – We saw small example of EOPF
- Parallelisation (Dask + xarray) + references at the end (local – coiled, etc)
- Other datasets but this will work similar whatever the format is (e.g. Earth Access LINK 1 and 2, CHIRPS, IMD)
- Vector data formats (geoparquet, etc)
- Deep Learning and xarray
- QGIS, GeoLibre (NEW!), Terrio (NEW!) also work with cloud-native
- Modular coding

Conclusion

- Cloud-native data formats: the need of the hour
- For a data scientist, 90% of our time goes in understanding and working with data
- Once you understand the data structure, everything pretty much leads to xarray
- Not only with Python but other programs access Cloud Native Data (QGIS, GeoLibre, etc)
  - But you will get the “horsepower” from Python only
- Best to go work with the original sources (e.g. ECMWF on CDS versus other sources Google for actual analysis, etc) – My opinion
- Keep on learning
- Happy mapping and coding and ENJOY !!

References

- https://tutorial.xarray.dev/overview/xarray-in-45-min.html
- https://ecmwf-training.github.io/c3s-training/submodules/c3s-training-submodule-reanalysis/reanalysis-heatwave/
- https://docs.xarray.dev/en/v2025.09.0/user-guide/dask.html
- Cloud Native Remote Sensing with Python course by Ujaval Gandhi Spatial Thoughts
- Mapping and data visualisation in Python with Ujaval Gandhi www.spatialthoughts.com
- thriveGEO (2026). Cloud-Native Geospatial 101 workshop notebooks. Cloud Native Geospatial Forum, Utah, USA, 6 October 2026.
- https://github.com/developmentseed/ingarss-workshop-2024
- https://www.technicalgis.com/tutorials/era5-python-xarray/
- Ryan Abernathey, CMIP6 in the Cloud Five Ways, Medium article
- https://tutorial.xarray.dev/overview/xarray-in-45-min.html

- https://eopf-toolkit.github.io/eopf-101/
- https://earthkit.ecmwf.int/
- https://guide.cloudnativegeo.org/overview.html#/cloud-optimized-geospatial-formats-overview
- https://guide.cloudnativegeo.org/overview.html#/what-is-zarr
- https://guide.cloudnativegeo.org/cloud-optimized-geotiffs/cogs-examples.html
- Earth Access Python library tutorial
- https://climate.copernicus.eu/work-progress-our-data-stores-turn-arco
- https://github.com/bopen/edh-learning/tree/main/notebooks
- https://zarr.dev/datasets/